MixBytes()

[SECURITY AUDIT REPORT]

Lido Lend Guardian Daemon SDK

PREPARED FOR LIDO LEND

REPORT DATE

November 09, 2026

[CONTENTS]

Table of Contents

1. Introduction 3
   1.1 Disclaimer 3
   1.2 Executive Summary 3
   1.3 Project Overview 5
   1.4 Security Audit Methodology 6
   1.5 Risk Classification 8
   1.6 Summary of Findings 9
2. Findings Report 10
   2.1 Critical 10
   2.2 High 11
      H-1 An event-dense history range aborts every cycle, with no adaptive splitting 11
   2.3 Medium 12
      M-1 Report submission saturation livelocks the cycle 12
      M-2 Cycle work is unbounded and non-incremental against a fixed attestation window 13
      M-3 Receipt evidence is not bound to the encoded registry call 14
      M-4 Reconstruction evidence reads lack canonicality validation 15
      M-5 Suffix-truncated history evades the position-without-history guard 17
      M-6 A long Retry-After pauses the cycle beyond every configured budget 18
      M-7 Nonce cancellation repeats the send's fee ladder and never outbids the stuck transaction 19
      M-8 Work reuse inherits a supply omitted by an earlier attempt's incomplete log read 20
   2.4 Low 22
      L-1 Confirmation depths are not validated against the 256-block blockhash window 22
      L-2 Pending-nonce false negative silently no-ops recovery 23
      L-3 Top-up oscillation forces repeated re-screening of the whole position 24
      L-4 A screening or signing call that never settles leaves the Guardian locks held forever 25
      L-5 With maxGuardiansQuorumDelay = 0 the daemon screens a supply only after it can already back a borrow 26
      L-6 With logQuorum, one silent provider slows every compared log read to the request timeout 27
      L-7 An administrative unban is not treated as a boundary for stale screening decisions 28
3. About MixBytes 29

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

Lido Lend is a lending protocol built on per-market controllers with delayed collateral activation: supplied collateral does not become borrowable immediately but enters an activation window during which Guardian daemons are expected to review it. The audited SDK implements one complete Guardian cycle - building a scan plan from chain state and the previous heartbeat, discovering candidate positions, reconstructing the contributing supplies from market lifecycle history, screening them through a caller-supplied screening provider, and submitting suspect/flag reports plus a coverage heartbeat whose digest commits the complete reconstructed anchor set to the on-chain `GuardianSignals` registry.

The interim security review was conducted over 9 working days via manual code review and a proprietary AI-assisted analysis tool.

Beyond the SDK's own cycle logic, the review weighed its interactions with the external systems it consumes - the on-chain `GuardianSignals` registry, the Lido Lend core and market-controller contracts whose events it reconstructs. The team also worked the standard internal checklist for off-chain components: input validation and fail-closed handling, ABI encoding/decoding parity with the pinned on-chain sources, authorization and cycle-boundary checks, integer widths and time-unit conversions, nonce and replacement discipline, and secret handling. Project-specific vectors derived from the audit scope are listed below.

- **Heartbeat over incomplete evidence.** Whether any read path can feed the coverage digest an incomplete or outdated work set without an error: unfenced range reads served by lagging or failover-routed endpoints, truncated history walks, and divergence between the hash-pinned discovery evidence and the range-query reconstruction of the same position.
- **Swappable-component substitution.** What a caller-supplied `RpcClient` or screening provider can make the SDK attest, given that the Guardian's confirmation checks execute inside the client and the returned receipt carries no binding to the encoded registry call.
- **Window-budget exhaustion.** Cheap borrower-side actions against the fixed 256-block attestation window: funded position counts against submission throughput, dust supplies inflating discovery, and top-up oscillation forcing re-screening, each aiming to withhold committee heartbeats until the `maxGuardiansQuorumDelay` fallback lets unscreened collateral activate.
- **Lifecycle-history misinterpretation.** Whether the backward reconstruction walk can misidentify a position's contributors through the supply/activation log pairing, dust absorption, and non-monotonic timers after drains and re-supplies, verified against the pinned market-controller sources.
- **Multi-provider read comparison.** The `logQuorum` union discipline: which endpoint divergences it rejects, which it accepts (providers that lag alike while a complete one does not answer), and whether its fail-closed

strictness can itself be used to withhold heartbeats.

The on-chain contracts the SDK encodes calls for and decodes events from - `LidoLend`, `LidoLendMarketController` with `CollateralActivationLib`, and the `GuardianSignals` registry - were outside the audit scope; they served as the pinned-commit reference (core `ec71716`, markets `bdf9e50`) for behavior-parity and encoding checks rather than as subjects of line-by-line review. Assessment of those contracts was limited to the SDK-facing surfaces the in-scope code actually consumes.

Below we set out our overall assessment, key assumptions, and main recommendations.

- **Strong architecture and separation of concerns.** The cycle is cleanly split into plan / discovery / reconstruction / screening / submission phases with immutable phase results and hash-pinned evidence reads on the discovery path.
- **Read-discipline asymmetries are the main risk area.** The strongest findings cluster where one read path validates evidence strictly (hash-pinned, removed-flag-checked) while a sibling path performing the same logical job does not: plan-anchor log selection, reconstruction ranges, and the mirrored supply/activation pairing. Uniform read discipline would eliminate the highest-severity issues.
- **Trust in the swappable client is unbounded at the Guardian boundary.** The confirmed-report precondition and heartbeat confirmation flow through the caller-supplied `RpcClient` with no boundary-level binding between the encoded registry call and the confirmed receipt; a boundary check would close the component-swap class.
- **Liveness under adversarial workloads is unbudgeted.** Report submission has no admission cap against the 256-block heartbeat window, and reconstruction re-derives entire histories each cycle with uncached supply reads - cheap borrower-side actions can withhold heartbeats or tax the daemon, and an event-dense history planted for gas costs alone stops every guardian outright (finding 1, High).
- **Exceptional test discipline.** The suite enforces 97% line / 94% branch / 98% function coverage in CI (455 tests, including Anvil-backed conformance runs against bytecode-pinned fixtures).
- `logQuorum` **fail-closed strictness is by design.** With `logQuorum` enabled, a log read is accepted only if enough providers returned the same answer and the accepted answer contains every log that any provider returned, so one persistently divergent endpoint - a node briefly following a minority fork, or a buggy or compromised provider - returning a single extra well-formed log fails every compared read with `answers-differ` and stops the cycle. This strict union is the intended anti-hiding guarantee: a heartbeat is deliberately withheld rather than attesting over an answer that may silently omit a log. The accepted operational consequence is that a divergent endpoint withholds heartbeats, so operators should monitor for persistent `answers-differ` stops and remove the offending endpoint.

1.3 Project Overview

Summary

CLIENT	Lido
CATEGORY	Lending
PROJECT	Lido Lend Guardian Daemon SDK
TYPE	TypeScript
PLATFORM	Off-chain (EVM)
TIMELINE	29.09.2026 - 03.11.2026

Scope of Audit

`src/core/guardian.ts` `src/core/outcomes.ts` `src/core/digest.ts` `src/core/index.ts` `src/core/stop.ts` `src/core/screening-provider.ts` `src/core/types.ts` `src/contracts/markets.ts` `src/contracts/guardian-signals.ts` `src/contracts/decode-logs.ts` `src/chain/viem-client.ts` `src/chain/viem-transport.ts` `src/chain/base-rpc-provider.ts` `src/chain/rpc.ts` `src/chain/log-order.ts` `src/events.ts` `src/common/immutable.ts` `src/common/validation.ts` `src/common/types.ts` `abi/GuardianSignals.abi.ts` `abi/MarketController.abi.ts` `abi/Core.abi.ts` `example/daemon.ts` `example/env.ts` `example/screening-adapter.ts`

Versions Log

DATE	COMMIT HASH	NOTE
29.09.2026	`549fca97b9d42d5f6f4f89c428a2da578426758a`	Initial Commit
23.10.2026	`b016f6cee6b4fd1b3d3a3bb218f248eb27f541b5`	Commit for Re-audit
03.11.2026	`b566e019684568e1d505ff25f77b3da4e61fe095`	Commit for Re-audit

1.4 Security Audit Methodology

Stage 1 Interim Audit

01 PROJECT ARCHITECTURE REVIEW

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING THE PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

- Understanding overall system design and service and module interactions
- Mapping responsibilities of each component and protocol workflows
- Conducting a thorough line-by-line review of the codebase and modules
- Tracing request and data flows, trust boundaries, and state transitions
- Identifying external integrations, RPC and API surfaces, and third-party dependencies
- Forming an independent architectural understanding directly from code before validating documentation
- Running the project test suite to observe behavior and encoded invariants

02 ADVERSARIAL CODE REVIEW

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

- Analyzing the codebase from an attacker's perspective: what can fail, not only what works
- Searching for edge cases, invalid assumptions, unexpected call and message sequences, and unsafe state transitions
- Attempting to violate protocol invariants and find where guarantees break
- Analyzing the authorization surface, input validation, and the encoding parity the on-chain side relies on
- Writing targeted tests, modelling adversarial scenarios, mutation testing, fuzzing, and PoC exploits
- Independent exploit-path discovery by each auditor to reduce anchoring bias
- Collaborative pair-auditing of high-risk code led by the Team Lead

03 SYSTEMATIC VULNERABILITY ANALYSIS

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

- Reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- Analyzing common off-chain attack classes: authentication and authorization bypass, input validation and deserialization, race conditions and idempotency, secret and key handling, dependency and supply-chain risk, encoding parity with the on-chain parser
- Using proprietary AI tooling to surface candidate issues across broader attack vectors
- Manually validating and triaging AI-generated candidates

04 CONSOLIDATION OF AUDITORS' REPORTS

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

- Cross-checking findings across auditors; consolidating and deduplicating issues
- Resolving discrepancies in severity assessments
- Using proprietary AI tooling to assist with report drafting and consistency checks
- Manually reviewing and finalizing the report narrative for client review

Stage 2 Re-audit

RE-AUDIT

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THE UPDATED CODE INTRODUCES NO NEW VULNERABILITIES.

- Confirming each remediation matches the original recommendation
- Documenting rationale for any unresolved findings
- Verifying modified logic and integration points remain secure
- Executing the test suite after remediation, targeting fixed areas
- Running AI tooling on the updated codebase

1.5 Risk Classification

Severity Level Matrix

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
LIKELIHOOD: HIGH	CRITICAL	HIGH	MEDIUM
LIKELIHOOD: MEDIUM	HIGH	MEDIUM	LOW
LIKELIHOOD: LOW	MEDIUM	LOW	LOW

IMPACT

HIGH - Loss or theft of user or protocol assets, compromise of keys, credentials or privileged access, or an attacker acting on behalf of users or operators.

MEDIUM - Partial or temporary loss of a security property, such as disruption that needs operator intervention, exposure of sensitive data, or incorrect outputs that do not directly move funds.

LOW - Limited security effect, such as degraded behavior that recovers on its own, with no assets or sensitive data at risk.

LIKELIHOOD

HIGH - Any external party can trigger it, or it arises under normal operating conditions.

MEDIUM - Needs specific conditions, such as an authenticated or trusted actor, a particular configuration, or a failing external dependency.

LOW - Needs privileged access to the system or its infrastructure, or an unlikely combination of conditions.

ACTION REQUIRED

CRITICAL - Must be fixed as soon as possible.

HIGH - Strongly advised to be fixed in order to minimize potential risks.

MEDIUM - Recommended to be fixed to improve security and stability.

LOW - Recommended to be fixed to improve overall robustness and efficiency.

FINDING STATUS

● FIXED - The recommended fixes have been implemented and the issue no longer impacts the security of the protocol.

● PARTIALLY FIXED - The fixes have been partially implemented, reducing the impact, but the issue has not been fully resolved.

● ACKNOWLEDGED - The fixes were not implemented; the finding is unresolved or was accepted by the team without code changes.

1.6 Summary of Findings

Findings Count

0	1	8	7	16
CRITICAL	HIGH	MEDIUM	LOW	TOTAL

Findings Statuses

All finding statuses reflect the state of the code at the latest revision listed in the Versions Log.

ID	FINDING	SEVERITY	STATUS
H-1	An event-dense history range aborts every cycle, with no adaptive splitting	HIGH	FIXED
M-1	Report submission saturation livelocks the cycle	MEDIUM	FIXED
M-2	Cycle work is unbounded and non-incremental against a fixed attestation window	MEDIUM	FIXED
M-3	Receipt evidence is not bound to the encoded registry call	MEDIUM	FIXED
M-4	Reconstruction evidence reads lack canonicality validation	MEDIUM	FIXED
M-5	Suffix-truncated history evades the position-without-history guard	MEDIUM	FIXED
M-6	A long Retry-After pauses the cycle beyond every configured budget	MEDIUM	FIXED
M-7	Nonce cancellation repeats the send's fee ladder and never outbids the stuck transaction	MEDIUM	FIXED
M-8	Work reuse inherits a supply omitted by an earlier attempt's incomplete log read	MEDIUM	FIXED
L-1	Confirmation depths are not validated against the 256-block blockhash window	LOW	FIXED

ID	FINDING	SEVERITY	STATUS
L-2	Pending-nonce false negative silently no-ops recovery	LOW	FIXED
L-3	Top-up oscillation forces repeated re-screening of the whole position	LOW	ACKNOWLEDGED
L-4	A screening or signing call that never settles leaves the Guardian locks held forever	LOW	FIXED
L-5	With maxGuardiansQuorumDelay = 0 the daemon screens a supply only after it can already back a borrow	LOW	ACKNOWLEDGED
L-6	With logQuorum, one silent provider slows every compared log read to the request timeout	LOW	ACKNOWLEDGED
L-7	An administrative unban is not treated as a boundary for stale screening decisions	LOW	ACKNOWLEDGED

[2 FINDINGS REPORT]

2.1 Critical

NO CRITICAL SEVERITY FINDINGS

2.2 High

H-1 An event-dense history range aborts every cycle, with no adaptive splitting

HIGH

FIXED

DESCRIPTION

Reconstruction reads a position's controller events in fixed ranges of `maxLogBlockRange` blocks, aligned to the core deployment block (base-rpc-provider.ts:165); a range is limited by blocks, not events, and the whole response must arrive before the backward walk can decode it and stop early. Providers cap the events or the response size of one `eth_getLogs` call (Infura, for example, documents 10,000 events), and the transport enforces its own response-size limit (viem-transport.ts:336). One range above the limit throws, and the exception aborts the entire cycle before screening and any report (guardian.ts:195); a retry reads the same range and fails again.

An unprivileged borrower can plant such a range around a real position: dust supplies below the activation threshold are withdrawable at once, so a helper contract can repeat supply and withdrawal, two controller events per cycle, and then start the activation in the same fixed range. With a 10,000-event limit about 5,000 dust cycles suffice, roughly 0.5 ETH of gas at 1 gwei and eleven dense blocks, with no capital locked. The events are on chain, so every guardian hits the same rejection, and nothing in the SDK routes around the dense range; the committee-wide stall then ends only at the `maxGuardiansQuorumDelay` fallback, where unscreened collateral can activate.

The failure is loud and nothing incomplete is attested, but the trigger is permissionless, capital-free, deterministic and permanent until the code changes, and because the poison lives in the on-chain history it disables the committee's review as a whole, with no redundancy to absorb it; we therefore rate the issue High.

RECOMMENDATION

> We recommend splitting a range that fails on event count, response size or timeout in half and retrying both parts, down to a single block, keeping completeness and order when merging, and failing loudly only when a single block cannot be read. Globally shrinking `maxLogBlockRange` also avoids the rejection but multiplies the request count of ordinary history.

CLIENT'S COMMENTARY

Client: Fixed in #9 (`bc7417e` , `06e95a9`) and #27 (`0b0cd1d`).

MixBytes: It is worth noting that range splitting currently triggers on any read error, not only size refusals. This is acceptable, as more detailed error filtering could miss an unexpected provider-specific refusal wording and reintroduce the finding. The only observable effect is a slightly delayed cycle failure on a total outage.

2.3 Medium

M-1 Report submission saturation livelocks the cycle

MEDIUM

FIXED

DESCRIPTION

Reports are sent one batch at a time, and the SDK waits for each batch to be confirmed on chain before sending the next. The heartbeat must be sent while the report block is still inside the 256-block blockhash window, so with the example configuration a single cycle has room for roughly 1,500 to 2,000 suspect reports. If the guardian has more due targets than that, the cycle runs out of time before the heartbeat and stops with `report-block-aged-out`. Nothing is saved between attempts, and the specification requires a fresh `suspect` report on every retry, so the next cycle sends the same reports again and fails in the same way. The guardian is stuck in a loop: it never sends a heartbeat, and it pays for all suspect reports again on every failed attempt. A repeated suspect is not a no-op on chain: the registry refreshes the suspect snapshot and emits the event again.

An attacker can create this state with about 1,000 positions funded, and fewer suffice when the positions are spread over several markets, because batches are grouped per market and per report kind. The same set of targets stalls every committee member at once. If enough guardians stall, the protocol's `maxGuardiansQuorumDelay` fallback (at most 7 days) lets the attacker's own collateral activate without any guardian review.

RECOMMENDATION

> We recommend broadcasting all batches of a kind with consecutive nonces and waiting for depth once (per the specification), adding a pre-submission window-fit check that fails with a dedicated reason instead of spending gas, and skipping re-sent suspects this guardian already recorded after the plan's previous heartbeat.

CLIENT'S COMMENTARY

Client: Partially Fixed in #9 (`08b2669` , `724f656` , `b39bddb`) and #27 (`48404a6` , `1420bdb`).

The SDK does not send a `suspect` again if this Guardian already recorded it after the previous heartbeat and it is still confirmed and inside `suspectWindow` (specification 0.3.0). When the provider's answer marks some anchors incomplete, the SDK sends the reports for the judged ones and withholds only the heartbeat. A retry sends only the reports that are not recorded yet.

MixBytes: We consider the finding fixed: a suspect recorded since the previous heartbeat and still inside suspectWindow is kept instead of re-sent, and own-flagged targets are excluded from screening, so a retry sends no duplicate reports. The unimplemented recommendation items are throughput optimizations rather than livelock conditions.

M-2 Cycle work is unbounded and non-incremental against a fixed attestation window

MEDIUM · FIXED

DESCRIPTION

The heartbeat must be sent while the report block is still inside the 256-block blockhash window, which is roughly 50 minutes at example settings. The work that must fit into that window grows with the total history instead of the new activity. Discovery makes one controller-log read for every supply in the 7-day window, including supplies that are already covered, and waits for each read before starting the next. Reconstruction re-reads each position's full history back to the deployment block.

No cache survives between attempts, so a failed attempt repeats all of this work from scratch. The window also ratchets: it moves only after a successful heartbeat, so each failed attempt must scan a larger range than the last, and a guardian that once overruns the window can no longer complete any cycle. Ten days of downtime, for example, leave about seventeen days of window, and at 100 ms per sequential request the classification reads alone exceed the 49-minute budget.

A busy market, a restart after downtime, or an attacker spreading many small supplies over distinct blocks can push every attempt past the window, and the guardian never sends a heartbeat; a supply below the activation threshold turns to dust that the borrower can withdraw at once, so one such supply per block costs only gas, and dense ranges fail outright on provider result-size limits instead of being split. With `logQuorum` enabled, every log read is additionally sent to every provider.

RECOMMENDATION

> We recommend reading controller lifecycle events per controller with windowed range queries joined locally by `(transactionHash, logIndex - 1)`, caching immutable historical evidence across attempts keyed by block hash, bounding read concurrency, and adding a per-cycle read budget that converts unbounded work into a visible deferral. We further recommend documenting that a new guardian's daemon should start right after `GuardianAdded`, and that operators should monitor cycle duration against the plan window.

CLIENT'S COMMENTARY

Client: Partially Fixed in #9 (`bc7417e` , `06e95a9` , `120c934` , `4630b46` , `cb8ae57` , `c041cc5`), #10 (`58b1424`) and #27 (`1420bdb` , `a5cda2c`).

Discovery reads the window in pieces. Each supply is paired with the controller event from the same transaction. The number of reads depends on the window length and the number of positions, not on the number of supplies. History walks run `historyReadConcurrency` at a time. A stale or incomplete attempt hands its work to the next one, which walks only the positions that changed. The guides state the attempt budget and tell operators to start right after `GuardianAdded` and to watch the cycle duration.

MixBytes: We consider the finding fixed: cycle work is now bounded by window length and position count rather than supply count, and a stale or incomplete attempt hands its work to the next one, which re-walks only the positions that changed. It is worth noting that discovery still re-reads the full window on every attempt, so the first cycle after long downtime remains heavy; this is documented in the guides, and we are satisfied with the residual.

M-3 Receipt evidence is not bound to the encoded registry call

MEDIUM

FIXED

DESCRIPTION

Before sending the heartbeat, the SDK trusts the caller-supplied `RpcClient` to confirm every report. The `ReceiptRef` type the client returns carries no `to` or `from` field, so the SDK can only verify that the named transaction hash exists and is confirmed, never that the transaction was the report it just sent. The only check that ties a receipt to the encoded call lives inside the shipped `ViemRpcClient` (viem-client.ts:131-133), which a custom client replaces. The Guardian's read interface is the client itself (`RpcClient` extends `RpcProvider`), so even the pre-heartbeat confirmation checks execute inside the same swappable component; a client that overrides them needs no real receipt at all.

A third-party client adapter, or a buggy custom client that swallows a send timeout and returns its last confirmed transaction instead of failing, passes the gate with a real but unrelated receipt. The SDK then sends the heartbeat and reports success while the missing reports never landed on chain, and the quorum frontier advances on unreported work. On chain, every guardian's suspect report independently extends the activation delay (`suspectCount * delayPerSuspect`), so a single silenced report is not absorbed by the committee: when this guardian is the only one whose screening flagged the borrower, no surcharge applies and the collateral activates unscreened.

The shipped client does not have this bug, and an operator who wanted to cheat could modify the SDK itself, so this is a boundary hardening gap.

RECOMMENDATION

> We recommend carrying `to` / `from` on `ReceiptRef` and re-fetching the receipt at the Guardian boundary before `#submitHeartbeat`, requiring `to == registry` and `from == configured signer`.

CLIENT'S COMMENTARY

Fixed in #12 (`a171c1a`, `3c4ebcc`) and #27 (`8fd65d8`, `191bc39`).

Before the heartbeat the SDK reads each report's receipt and transaction again. The transaction must come from the signer, go to `GuardianSignals` and carry the report's calldata.

M-4 Reconstruction evidence reads lack canonicality validation

MEDIUM

FIXED

DESCRIPTION

When the SDK rebuilds a position's history during reconstruction, it reads logs by block-number ranges and never checks that the answering node is at the current chain head. Other phases of the cycle do check this: they verify the report block's hash before and after their reads.

A load-balanced RPC endpoint, or the transport's failover to a second configured URL, can route these reads to a node replica that is a few blocks behind. The failover route is easy to reach: without `logQuorum` the transport is viem's fallback, which starts every request at the first endpoint and moves to the next one only when a request errors, and a range log read often errors for rate, range or result-size limits; only that read then goes to the second endpoint, which may lag or follow another branch and answer without the newest logs and without an error, while the block-header checks still succeed on the first endpoint. Discovery's range reads share the same exposure.

The replica clamps the requested range to its own head without raising an error, so the answer silently misses the newest events of a position. Reconstruction then rebuilds an outdated state for that position with no error and no stop, and once the node catches up every later check passes, so the heartbeat is sent over an incomplete work set. The miss is permanent: once that heartbeat lands, the next cycle's coverage filter sees the position's deadline at or before the heartbeat's block and treats it as already covered, so the supply is never screened.

A fully empty answer fails closed with `position-without-history`; the silent paths are a partially truncated history, which makes the walk rebuild an outdated state, and a truncated state whose activation deadline has not yet passed, which drops the position from the work set.

With `logQuorum` enabled the compared log path accepts the quorum's complete answer and merely reports the lagging provider, so this vector is covered for log reads, but the option is runner policy and off by default, and providers that lag alike still form a quorum while a complete one does not answer.

RECOMMENDATION

> We recommend bracketing reconstruction reads with the same canonicality check discovery applies (re-check the report block's hash before and after the walk, stopping on drift), and verifying that the newest returned log's `blockHash` is canonical. We further recommend requiring the example application to enable `RPC_LOG_QUORUM` whenever several endpoints are configured, and documenting that the endpoints must belong to independent providers.

CLIENT'S COMMENTARY

Client: Partially Fixed in #10 (`01418a8` , `e5dcc3b` , `58b1424`), #15 (`a7cadfb`) and #27 (`48404a6` , `14ae820`).

The SDK checks the report block before and after the history walks. The newest event of each walk must be in the canonical block. The guides explain what only `logQuorum` can catch and recommend independent providers.

MixBytes: We consider the finding fixed: the report block is checked before and after the history walks, and each walk's first event must equal discovery's newest event and sit in the canonical block at its

number. It is worth noting that an omitted supply and a reused position remain detectable only with logQuorum, which stays optional runner policy; the guides recommend at least three independent providers with a quorum of two for production, and we accept this as a deployment requirement.

M-5 Suffix-truncated history evades the position-without-history guard

MEDIUM

FIXED

DESCRIPTION

Discovery identifies a candidate with a strict read pinned to a block hash, but reconstruction then rebuilds the same position from plain block-range queries. The guard `position-without-history` (markets.ts:295-301) fires only when the range query returns no events at all. If the node's log index is behind and returns some older events while missing the newest ones, the guard passes, the walk sees only an old state of the position, and the position is quietly dropped from the work set with no error. The drop is reachable because a position's deadline is not monotonic over its history: a full drain clears the timer and a later re-supply restarts it, so a walk truncated before the re-supply sees only the drained state, whose deadline has not started. Truncation can also leave outdated anchors in the work set or stop visibly with `position-history-broken`; the silent drop is the worst of the three outcomes. The SDK held the correct, newer information from the discovery step but discards it before reconstruction, so the two results are never compared, and the heartbeat commits a digest that omits the position.

RECOMMENDATION

> We recommend passing the discovery-time classification evidence into `reconstructWork` and stopping with `position-history-broken` when the walk's newest event is older than the discovery-time classification block.

CLIENT'S COMMENTARY

Fixed in #10 (`e5dcc3b`).

M-6 A long Retry-After pauses the cycle beyond every configured budget

MEDIUM

FIXED

DESCRIPTION

viem computes the pause between HTTP retries from the response itself: an HTTP error with a numeric `Retry-After` header is retried after exactly that many seconds, overriding `retryDelayMs` and every other configured delay. A provider answering one read with `429 Too Many Requests` and `Retry-After: 3600` therefore parks the daemon inside a single RPC call for an hour. Neither the per-attempt HTTP timeout (daemon.ts:39) nor the `sendTimeoutMs` budget interrupts the pause, because all deadlines in the client and in the shared confirmation wait are checked between operations, never during one (guardian.ts:750-776 checks the remaining budget only after `getBlock` returns). The example enables the header path by default (`RPC_RETRY_COUNT = 2`), and its scheduler cannot even retry, because it awaits the cycle (daemon.ts:75).

Rate limiting is an ordinary provider condition, so a single response can stall reports and the heartbeat for the length of the header. With several fallback endpoints the pause needs every endpoint of the pass to fail and the last one to carry the header; a single-endpoint setup, common for smaller operators, is exposed directly. The stall is liveness only: when control returns, the aged report block stops the attempt and no incomplete work is attested.

RECOMMENDATION

> We recommend clamping the retry delay derived from `Retry-After` to the remaining operation budget before it reaches viem, and making the deadlines effective during RPC calls by passing an abort signal with the remaining time through the transport. The pause itself should be visible to the operator.

CLIENT'S COMMENTARY

Client: Partially Fixed in #13 (`6d6c88b`).

The transport caps `Retry-After` at `timeoutMs` before viem sees it. The capped value is reported in `rpc.requestFinished` as `retryAfterMs` .

MixBytes: We consider the finding fixed: every Retry-After pause is now bounded by the request timeout and reported as retryAfterMs. It is worth noting that under a sustained rate limit the clamped pause retries inside the ban window, and range splitting then repeats these futile requests ~log2(range) times; this is acceptable with the recommended configuration, where fallback moves the read to another provider and logQuorum treats a rate-limited provider as absent.

M-7 Nonce cancellation repeats the send's fee ladder and never outbids the stuck transaction

MEDIUM

FIXED

DESCRIPTION

When a send exhausts its attempts and leaves a transaction pending, that last broadcast already carries fees bumped once per attempt. `clearPendingNonces` then prepares each cancellation from a fresh estimate and bumps only against its own previous cancellations (guardian.ts:443-470: the first cancellation uses the estimate unchanged), so the recovery repeats the same fee sequence the send used. With unchanged estimates the last cancellation ties, not exceeds, the pending transaction's fees, and the node rejects every attempt as an underpriced replacement; the example's policy of five attempts with a 10% bump (daemon.ts:45-49) ends with `Nonce clearing exhausted sendAttempts`. The example then terminates the process, and a restart repeats the sequence, so the daemon stays down until fees move or an operator clears the nonce by hand.

The recovery thus fails deterministically in exactly the situation it exists for: a stuck send under flat fee estimates. The failure is loud, no incomplete work is attested, and if the estimates rise by one bump step the last cancellation does replace the pending transaction.

RECOMMENDATION

> We recommend basing the first cancellation on the fees of the last broadcast for that nonce, taking the maximum of the current estimate and the pending transaction's fees increased by the bump, and keeping later cancellations on the same basis. The fees of the last broadcast per nonce can be retained by the client for the recovery, with a persisted fallback for transactions that predate a restart.

CLIENT'S COMMENTARY

Client: Fixed in #14 (`4ec7a3e`).

If the node rejects a cancellation, the SDK raises its fee at once and sends it again, until the node accepts it or the fee would exceed `maxFeePerGas`.

MixBytes: It is worth noting that every broadcast-stage failure - a gas price rejection or a network error alike - triggers an immediate fee bump, so an unreachable broadcast path ratchets the cancellation fee up to maxFeePerGas. This is acceptable, as a re-broadcast must outbid the possibly landed transaction under the replacement rules, and a mined cancellation is not repriced. The only observable effect is a cancellation included at a higher fee than necessary after network-side broadcast failures, bounded by maxFeePerGas and the 21000 gas of a self-transfer.

M-8 Work reuse inherits a supply omitted by an earlier attempt's incomplete log read

MEDIUM · FIXED

DESCRIPTION

A `stale` or `incomplete` attempt hands its reconstructed work to the next attempt, which reuses a position's anchors instead of re-walking its history when the newest lifecycle event that discovery read for the position is not after the earlier work's report block (markets.ts:194-218, the check at L205-L209). This treats the earlier work's report block as a proxy for "the earlier attempt saw every event through this block". The proxy holds only if the earlier attempt's log reads were complete - which is exactly what an incompletely answering endpoint breaks. As documented in the README, a load balancer can answer the head and block reads from a fresh backend while `eth_getLogs` is served by a lagging backend that returns logs only up to its own head without an error.

The scenario: attempt 1 runs against such an endpoint. The plan's report block `R1` comes from the fresh backend, but log reads omit a top-up supply at block `B <= R1` for a position that also has an older in-window supply. The position is reconstructed with the old anchors only, and the attempt ends `incomplete` or `stale`, carrying the work with `reportBlock = R1` into the next attempt (daemon.ts:123-134). Attempt 2 runs against a healthy endpoint: discovery re-reads the window and does see the top-up, so the candidate's newest event sits at `B`. Because `B <= R1`, the reuse check still passes, the position is not walked, and the earlier anchors are kept. The top-up is never screened, yet the heartbeat at the new report block `R2 >= B` attests coverage past it.

Without the reuse, the next attempt is self-healing: a full reconstruction finds the missed supply and screens it before any heartbeat. Reuse converts a transient omission into a permanently attested gap. This is not the residual the README documents for reuse - that text covers a current attempt whose discovery is lied to, while here the current attempt's reads are complete and correct, and the claimed invariant that with complete reads the work equals a full reconstruction is violated. The scenario is reproduced with the repository's test harness: attempt 1 with log reads filtered below the top-up block ends `incomplete` with only the old anchor; attempt 2 with complete reads and reuse attests after asking the screening provider about the old anchor only, while a full reconstruction at the same head asks about both.

The recommended production configuration prevents the scenario from starting: with logQuorum over at least three independent providers, a single lagging provider does not shrink the answer - the quorum returns the complete logs and reports the difference - and two providers lagging alike are rejected when the third returns a log they lack, so attempt 1 either sees the supply or fails without building work. Only a log that every answering provider omits escapes the comparison, which is the documented limit of the quorum. Setups without it, which the guides describe as unsuitable for signal mode, are exposed.

RECOMMENDATION

> We recommend recording in `Work`, for each position, the `LifecycleEventRef` its walk started from, and reusing the position's anchors only when that reference equals the `newestEvent` discovery read in the current attempt (block number, log index and block hash); any other position is walked again.

CLIENT'S COMMENTARY

Client: Fixed in #34 (`1f6b5c9`).

Additionally, the README now requires `logQuorum` of at least 2 when an endpoint can answer a log range beyond its head without an error.

The same pull request adds two checks of the history walk beyond this finding. The walk stops when two consecutive lifecycle events do not match the controller's writes (`2030488`). It also stops when an activation's amount is not its paired supply plus the dust before it (`786a4af`).

2.4 Low

L-1 Confirmation depths are not validated against the 256-block blockhash window

LOW

FIXED

DESCRIPTION

At setup, `heartbeatReserveBlocks` and `reportBlockLag` are checked against the 256-block limit, but the confirmation depths `txConfirmationBlocks` and `interBatchConfirmationBlocks` are only checked to be at least 1. A setting that looks reasonable, such as `txConfirmationBlocks = 128`, needs more blocks than the window can hold once two report kinds and the heartbeat are included. Every attempt then sends its reports, runs out of time and ends with `report-block-aged-out`, and every retry re-sends the suspect reports. With such a configuration, any cycle in which both report kinds are due can never complete; only single-kind cycles still fit.

RECOMMENDATION

> We recommend rejecting configurations where `reportBlockLag + 2 * txConfirmationBlocks + inter-batch waits + heartbeatReserveBlocks + margin > BLOCKHASH_LOOKBACK`, and documenting the resulting maximum confirmation depth.

CLIENT'S COMMENTARY

Client: Partially Fixed in #16 (`ee4cdfc`).

`txConfirmationBlocks` is capped at 64 and the cap is documented. Whether the lag, two depths and the reserve fit the 256-block window together is the Guardian's responsibility when configuring the application.

MixBytes: We consider the finding fixed: the 64-block cap on txConfirmationBlocks and the sendTimeoutMs minimum make the reported overshoot scenario statically impossible; the full formula check was not implemented, and the rationale is documented.

L–2 Pending-nonce false negative silently no-ops recovery

LOW

FIXED

DESCRIPTION

When the SDK sends the first transaction of a cycle, the viem library picks the nonce by asking one endpoint for the account's pending transaction count, and the recovery function `clearPendingNonces` uses the same single-endpoint signal to detect a stuck transaction, even though the transport can switch between several URLs. If the answering endpoint does not see the pending transaction (some providers ignore the pending tag, or the transaction was dropped from their mempool), the recovery silently does nothing: no event, no error. The next send reuses the still-live nonce, the two transactions collide, and delivery stays stuck with nothing in the logs to explain why. The harm is that reports and heartbeats stop being delivered.

RECOMMENDATION

> We recommend probing the pending nonce across transport endpoints and emitting an explicit uncleared/uncertain event from `clearPendingNonces` instead of returning void.

CLIENT'S COMMENTARY

Client: Partially Fixed in #14 (`3d11d34`).

`clearPendingNonces` emits `nonce.checked` with the `latest` and `pending` transaction counts before each decision.

MixBytes: We consider the finding fixed: the nonce.checked event with both transaction counts makes a silent no-op recovery visible before every clearing decision. Cross-endpoint probing of the pending count was deliberately not implemented; the residual is documented in the example guide, and we are satisfied with it.

L-3 Top-up oscillation forces repeated re-screening of the whole position

LOW

ACKNOWLEDGED

DESCRIPTION

Each time a borrower tops up a position, the activation deadline moves to a later time. When that deadline passes, the freshness filter resets and the SDK screens all of the position's contributors again, including those already screened and covered by earlier heartbeats. Banned borrowers can keep this up forever, because supplying is not blocked by a ban, and the extra work consumes committee screening capacity and feeds the saturation loop. For a target this guardian has already flagged, the provider call is skipped and the anchors only enter the digest, so a banned borrower's oscillation persistently costs discovery, reconstruction and digest work on every committee member; full re-screening with fresh suspect reports applies while the target is not yet flagged.

RECOMMENDATION

> We recommend tracking the guardian's prior coverage boundary per position (or accepting a caller-supplied prior-digest memo) so anchors already covered by an earlier heartbeat are not re-screened merely because a top-up extended the deadline.

CLIENT'S COMMENTARY

Acknowledged. Outside a deliberate attack this is rare, and the attack costs collateral: every round adds at least `minActivationAmount` to the position's `activating` balance, and withdrawing it ends the oscillation. A coverage boundary per position would make the digest depend on earlier heartbeats, so monitoring could no longer rebuild it from the previous one.

L-4 A screening or signing call that never settles leaves the Guardian locks held forever

LOW

FIXED

DESCRIPTION

The Guardian locks (`#activeCycle` , `#isWriting` , `#isSending`) are released only in `finally` , so they stay held while an awaited call is pending, and two awaited calls have no time bound: the screening provider call (guardian.ts:309) and the transaction signature, whose deadline is checked only after the signature returns (viem-client.ts:107-109, and unbounded again in `broadcastTransaction` at viem-client.ts:204). A vendor screening API without a timeout, or a remote signer (KMS, custody API, hardware wallet) that stops responding, therefore blocks the guardian indefinitely: the example application awaits the cycle without a watchdog (daemon.ts:77), heartbeats and reports stop with nothing in the logs, and a runner that abandons the attempt cannot reuse the instance, because every later `cycle()` fails with `another cycle is in progress` until the process is restarted. Nothing unscreened is attested, so this is a liveness issue only.

RECOMMENDATION

> We recommend bounding the signature by the remaining send deadline, including in `broadcastTransaction` , and bounding the screening call with a timeout whose expiry is treated as `screening-result-incomplete` . Clearing the lock flags on a timer is not a safe alternative, because the blocked operation may still resume.

CLIENT'S COMMENTARY

Fixed in #13 (`6d6c88b` , `a9bc9a8`) and #27 (`1420bdb` , `133235d`).

After `screeningTimeoutMs` every requested anchor is incomplete and a late answer is dropped; the attempt ends as `incomplete` without a heartbeat.

L-5 With maxGuardiansQuorumDelay = 0 the daemon screens a supply only after it can already back a borrow

LOW

ACKNOWLEDGED

DESCRIPTION

Discovery passes a supply to the screening provider only once its base activation deadline is at or before the report block (markets.ts:270, markets.ts:299). This ordering relies on the controller's quorum-frontier gate, where a lot matures only after the frontier, advanced by heartbeats, reaches its unlock time. The gate is optional: with `maxGuardiansQuorumDelay = 0`, maturity no longer depends on heartbeats at all (`CollateralActivationLib` of the pinned markets commit `bdf9e50`), and zero is the value in every market configuration of the deployment scripts (for example `script/config/markets/weth-wsteth.toml`, with base delays of 10 minutes to 2 hours) and in the SDK's conformance fixture (protocol.ts:90). A borrower can then borrow at the base deadline while the daemon first sees the supply in a later cycle, so its `flag` lands after the loan, and a ban does not unwind it. The activation delay exists precisely to let guardians flag before collateral becomes borrowable, so a zero value inverts the intended order.

The issue is rated Low because the activation parameters are marked as draft in the deployment scripts and a zero value is not expected in production; neither the specification nor the SDK states or checks the precondition, however.

RECOMMENDATION

> We recommend requiring a nonzero `maxGuardiansQuorumDelay` above the maximum activation delay for every market this daemon serves and stating that precondition in the specification, or screening supplies when they enter `activating` and submitting the required reports before the base deadline. The daemon could additionally read each served controller's activation config and stop when the frontier gate is disabled.

CLIENT'S COMMENTARY

Acknowledged. The specification now says what a zero `maxGuardiansQuorumDelay` means and that the daemon screens such a market like any other (#11 `fb748ad`, #27 `48404a6`). The daemon does not read this value, because its duties do not depend on it. A nonzero `maxGuardiansQuorumDelay` in the markets this daemon serves is the deployment's responsibility; the zero values in the deployment scripts are drafts.

L-6 With logQuorum, one silent provider slows every compared log read to the request timeout

LOW

ACKNOWLEDGED

DESCRIPTION

The compared log read asks every configured provider and waits for each to answer or time out (viem-transport.ts:133-142); awaiting all is documented policy, because a late answer may hold a log the quorum lacks. The budget consequence is that one provider which accepts requests and never answers before the timeout adds roughly `timeoutMs`, ten seconds in the example, to every `eth_getLogs`, and discovery performs its log reads one at a time (markets.ts:119-130). With a heartbeat a day old and a few hundred supply-carrying blocks in the window, the accumulated waits alone can exceed the 49-minute plan budget, the attempt ends with `report-block-aged-out`, and every retry repeats the reads because nothing is cached between attempts. Two healthy providers cannot mask the silent one, because the answer is returned only after every provider settles.

The daemon stops safely and nothing incomplete is attested, and removing the silent provider restores operation, which keeps the issue at Low.

RECOMMENDATION

> We recommend reusing classification reads across attempts keyed by block hash, bounding the additional wait after a quorum has answered, or excluding a provider that times out repeatedly within one attempt. Returning the first quorum answer early trades away the completeness guarantee of awaiting every provider, so that choice should be explicit and documented.

CLIENT'S COMMENTARY

Acknowledged. The behaviour is intentional: transport waits for every provider so that no log is missed. Since #9, discovery reads logs per window piece and controller instead of once per supply, and history walks run `historyReadConcurrency` at a time. A provider that keeps timing out should be removed from the configuration.

L–7 An administrative unban is not treated as a boundary for stale screening decisions

LOW

ACKNOWLEDGED

DESCRIPTION

An `unban` on the registry clears the ban and every guardian's flags. A guardian that was down later screens the supplies accumulated while it was away, and its own-flag check covers only the report block, so an unbanned target is screened again. The SDK neither passes the unban to the provider nor requires the provider to weigh it: if the provider still returns `flag` on the same evidence that a manual review already dismissed, the guardian flags again, and with enough such late flags, or a committee quorum of one, the user is banned again without a new supply or a new violation. The harm is a wrongful restriction that another unban can lift, and the screening decision itself belongs to the provider, which keeps the issue at Low.

RECOMMENDATION

> We recommend defining in the specification how an administrative unban affects earlier screening decisions, and passing the unban event, or a re-review hint, with the screening request, so a late flag requires fresh evidence.

CLIENT'S COMMENTARY

Acknowledged. The behaviour is intentional. The SDK screens every supply and does not set the provider's policy: the provider decides whether a supply is reported, with or without an earlier unban. A report that a manual review dismissed is handled in the Guardian's secondary review, which the specification leaves to the Guardian.

[3 ABOUT MIXBYTES]

Built on Trust

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems. The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle - from early architecture design to production audits and ongoing protocol evolution. Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including Lido, Aave, Curve, 1inch, OKX, Mantle, Fluid, Gearbox, Resolv, and others. To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review.

SECURED TVL	AUDITS DELIVERED	WEB3 PROTOCOLS
$50B+	300+	80+

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

Web-3 Design Review

Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.

AI Tooling

AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.

Smart Contract Audit

Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.

Security Support

Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

CONTACTS

hello@mixbytes.io